

An Analysis of Various Digital and Analog Systems

Kevin Worley and Justin Roberts
AET5420 Project 1

Table of Contents

Kevin Worley & Justin Roberts

Introduction	2
Methods	2
Test Signal Generation Process	3
Digital Experiment Design Process	3
Analog Experiment Design Process	4
Pictures of Digital Experiment EQ Settings	4
Mod Delay III Plugin Setting	5
Pro Tools Session Screenshot	5
Pictures of Analog Experiment	6
Room IR measurement Experiment	6
Hardware EQ Setting 1	6
Hardware EQ Setting 2	6
Pro Tools Screenshot	7
Data Analysis/IR Creation Using Matlab	7
Introduction	7
Digital EQ/Delay/Hardware EQ Analysis Procedure	7
Figures Part 1	8
Figure 1	8
Figure 2	9
Figure 3	9
Figure 4	9
Room IR Analysis Procedure	10
Figures Part 2	10
Figure 5	10
Figure 6	11
Figure 7	11
Figure 8	11
Figure 9	11
Conclusions Reached	12
Improvements to Experimental Design	12
Appendix	13
Matlab Code	13
Part 1 Code	13
Part 2 Code	15

Introduction

Our task for this assignment was to take a variety of Acoustical Measurements-both in the Digital and the Analog Worlds- of various audio equipment (and rooms) at various different settings/types. More specifically, we analyzed the usage of two digital plugins-Mod Delay III, and 7 band-EQ- in the digital domain. Within the analog domain, we performed IR's (Impulse Responses) for a Home Studio-in Justin's Apartment - as well as for the Hardware EQ unit (Pultec EQP - 1A).

In order to do this in the Digital Domain, the main vessels we utilized for this experiment on the Digital Side involved Pro Tools, an Impulse (generated using Matlab), a sample Vocal.wav file (to use as a sample audio file in the experiment), and Matlab for the analysis of our results.

In order to complete this in the Analog Domain, the main vessels we utilized were Pro Tools, a Sine Sweep (generated from Matlab using the chirp function-10 seconds long with fade in and fade out), an Audio Interface (Motu 828x), an Audio Speaker (Yamaha HS8), and a Condenser microphone (Rode NT1). We performed our Impulse Responses at Justin's Home Studio on Tuesday, April 2nd, 2024.

Our purposes of this experiment was to take the resulting audio prints of these and make them into IR's for both locales and analyze them (to make sure that A-The hardware/room/plugin worked the way it's supposed to and that B-to see if our impulse responses we derived in Matlab worked right, and were exactly the same compared to the actual prints captured in the experiment).

Methods

In our performance of this task, there were a variety of considerations that were made on our part in terms of the order of steps as they were performed for proper execution of the experiment. Firstly, we generated the Impulse (top of Part 1 Code), and the sine sweep/chirp (top of Part 2 Code) that we would then use for their respective parts of the experiment (Impulse for Digital, and the Sine Sweep for Analog). Once we generated these signals, we saved them as audio wav files using the audiowrite function in Matlab.

Test Signal Generation Process

Our Impulse Signal consists of a 1-Second Impulse (F_s -Sample Rate- of 48k) where the first sample was the impulse followed by 48K samples of zeros (using the zeros function in Matlab).

Our Chirp/Sine Sweep signal was generated using the Matlab Function "Chirp" in Matlab. It was a 10 second auditory sample ($F_s=48k$), with a one second linear fade in (linspace) applied at the beginning, and a one second linear fade out (linspace) applied at the end.

Our vocal file that we used for this test was an audio file that was provided to us by Dr. Tarr. It is a sample of a recording of a Male Country Vocalist that lasts approximately 25 seconds in length ($F_s = 48K$).

In order to maintain maximum consistency, we decided to generate all of our files (both files generated in Matlab as well as printed through Pro Tools with a Sampling Rate of 48K and 24 Bit bit depth.

Digital Experiment Design Process

Once we had our test signals generated, the next step of the process was to perform our tasks in PT (Pro Tools). In doing this, we started by creating 4 tracks in PT-two tracks with our Impulse signal, and two tracks with our vocal file. In the first track with the Impulse, we ran it through our Mod Delay III plugin with a medium delay (335 ms delay, with a feedback percentage of -50%). In the second Impulse Track, we ran it through our 7-band EQ plugin (settings were a HPF-High Pass Filter at 300 Hz, a dip at 2K with a Q factor of 4.22-negative 6.9dB, and a shelf at 12 kHz with a Q factor of 1 and a gain of +2.8 dB). These two plugins were chosen to run this portion of the experiment since they are plugins that come with Pro Tools. We ran our two Vocal Tracks through these same settings-one with just the Mod Delay III plugin, and the other running just through the 7-Band EQ plugin. Once these tracks were generated on Pro Tools, we created prints of these tracks (inputted and labeled as such below these files on our PT session) using the audio bounce feature in PT. As these prints were performed, we made sure to hide and deactivate all tracks that were not the desired track we were printing as we went through the process. After getting the prints, we performed some quality assurance measures to ensure that the prints we gathered sounded like they were supposed to be (pictures to follow below).

Analog Experiment Design Process

For the Analog Portion of our experiment, we chose to perform our tests in Justin's home studio on 4/2/2024. We utilized the required materials, a Pro Tools Session with our imported sinesweep.wav generated in MATLAB, the vocal track from part 1, a Rode NT1 Condenser Microphone, a Yamaha HS8 speaker, an audio interface(Motu 828x) and the Yamaha MGP24X console as our Mic pres (using the Mic Pres on the Board). We set up our speaker near the middle of the room (6 ft in front of the Rode NT1 and ran it through an input on the console, and set up our Microphone in the middle of the room using another input on our console. We then ran our Sine Sweep a couple of times through the room and printed the resulting signal using the record feature on PT. Once we performed this step, we ran our Sine Sweep and the Vocal wav file through our Hardware EQ - Pultec EQP - 1A (settings shown below; used two differing settings) and printed this signal onto its own track in Pro Tools as well.

Pictures of Digital Experiment EQ Settings



Mod Delay III Plugin Setting



Pro Tools Session Screenshot



Pictures of Analog Experiment

Room IR measurement Experiment



Hardware EQ Setting 1



Hardware EQ Setting 2



Pro Tools Screenshot



Data Analysis/IR Creation Using Matlab

Introduction

After performing our tests, we moved all of our resulting audio files (the prints that we got from both the Digital and Analog tests), into our directory folder in Matlab. With these files that we attained, there was a 2 part procedure that was ultimately decided on for the analysis of the data. The first thing we set out to do was to analyze the signals to make sure that we could visually see a difference between the dry signal (the original with no processing) and the wet signal (our printed files). This was done for all of the Digital FX tested in addition to the Hardware EQ tested (in terms of the graphs and general steps that we ran through to reach the final outcome). The second part of our experimentation involved deriving our H_n to then be able to use it as a convolution reverb in our IR. Once we got this H_n , we used the built in Matlab function (conv.), and ran our original vocal track through the function and performed a Null Test to see how accurate (or not) our resulting IR was.

Digital EQ/Delay/Hardware EQ Analysis Procedure

In summary, we first uploaded the impulseSignal.wav. The Impulse vs Delayed Impulse Response section reads the generated impulse signal and the digital delayed

ImpulseDelayPrint.wav from our Pro Tools session. The mono channel is extracted from the DVocalMono.wav. and calculated convolution of the delayed impulse with the original impulse signal using the built-in MATLAB function and played back the signal to hear the differences with the digital delay effect . Similar to the previous section, the Vocal vs Delayed Vocal portion compares the original vocal signal with its digital delayed version from Pro Tools. The difference between the convolved signal and the digital delayed vocal signal is then calculated. The Impulse EQ vs No EQ section examines the impact of digital EQ on the impulse signal. The digital EQ-applied impulseEQPrint_1.wav from Pro Tools compares its frequency response to that of the original impulse signal using the freqz function. We then calculated the convolution of the EQ-applied impulse with the original impulse and played back to hear , and the difference between the signals. The Vocal EQ vs Dry Vocal part contrasts the frequency response of a dry vocal signal with that of a digital EQ-applied VocalEQFXPrint____.wav from Pro Tools . The convolution of the digital EQ vocal with the original vocal is calculated and played back to hear the difference between the signals. Overall, this code is useful for analyzing the effects of digital delays and equalization on audio signals by visually and audibly comparing the original and processed signals.

Our analysis comparing our convolution to our prints showed that while the impulses didn't match perfectly, the only difference between the two was a very miniscule difference of .0002. The audio files sounded really distorted and clipped, this could be due to our files not being the same size, because of that there is probably some kind of delay within the creation of the convolution that is causing it to be off a lot more compared to the Impulse Response.

Figures Part 1

Figure 1

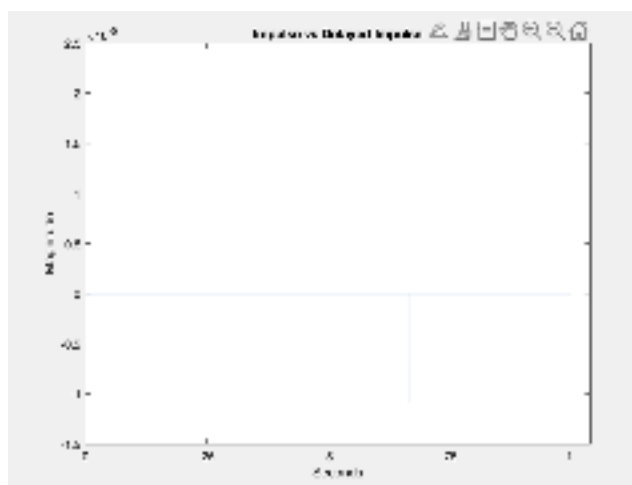


Figure 2

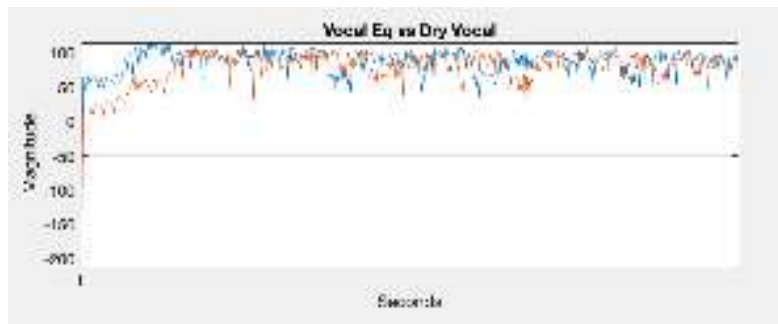
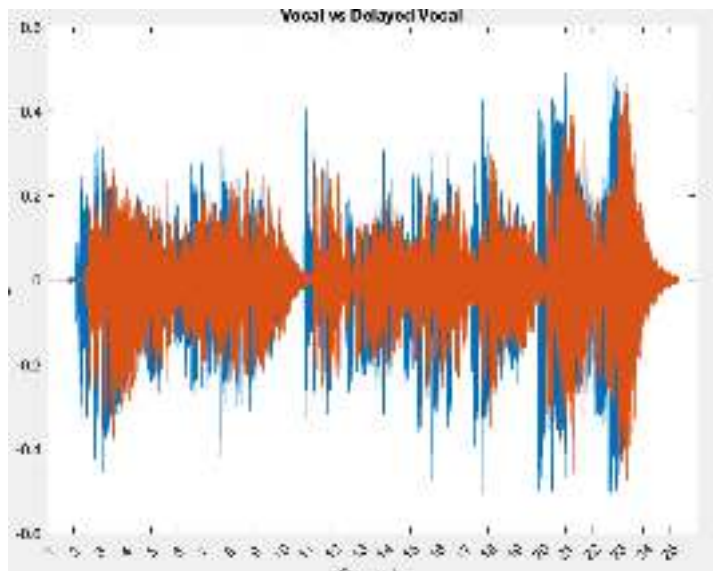


Figure 3



Figure 4



Room IR Analysis Procedure

In summary, The code starts by loading in the sinesweep.wav that was generated using MATLAB. The code zero pads the longer signal to match the length of the shorter one. We calculated the lengths of the sine sweep signal (sizeX) and the room response (sizey). We zero-padded the shorter signal to match the length of the longer one. After the signals are equal in length, we utilized the Fourier transforms of both signals using the fft function. Then, we calculated the conjugates of the Fourier-transformed signals. A for loop was utilized to set up a regularization parameter and pre-allocates arrays (freqBin) and (ampBin). It then repeats through a specified number of frequency bins (N), calculating the transfer function (Hk) for each bin. The transfer function is calculated differently depending on whether the product of the conjugates (z) is zero or non-zero. The results are stored in the HkBin array. The IFFT (HkBin) is calculated to obtain the impulse response in the time domain (hReal). Last, the script normalizes the impulse response.

Our analysis showed that the audio files sound distorted/clipped and showed similar characteristics/ issues to the digital vocal. The Impulse Response's error is probably due to a MATLAB syntax issue in the for loop. The Hardware EQ graph errors are similar to the audio file errors in part 1, and is also either a matlab syntax error, or an error in how we got our Impulse Responses of these files.

Figures Part 2

Figure 5

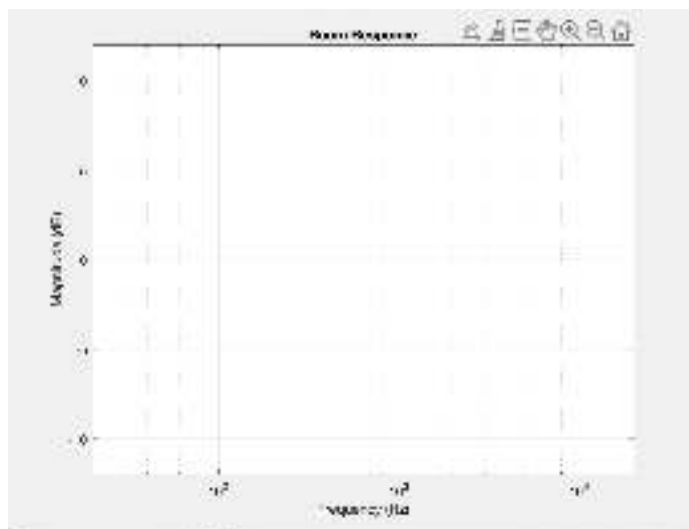


Figure 6

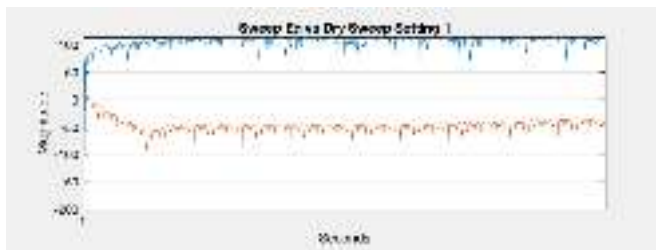


Figure 7



Figure 8

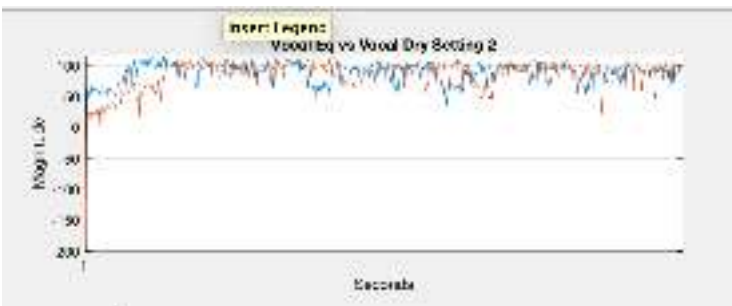
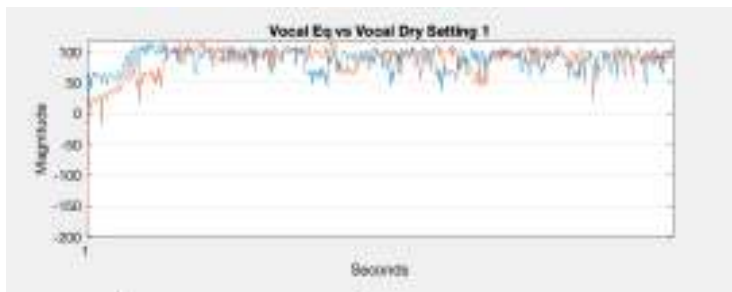


Figure 9



Conclusions Reached

Overall, we were able to capture a sine sweep of the room using both digital and analog equipment in a home studio. This is a crucial step in understanding the acoustic characteristics of the space. By sweeping through a range of frequencies, we were able to identify resonant frequencies, standing waves, and other acoustic anomalies that may affect recording and mixing in the space. The use of digital equipment, such as a computer with audio analysis software, provides precise control and detailed analysis of the frequency response. On the other hand, using analog equipment such as the Pultec EQP - 1A and an affordable quality microphone, Rode NT1, adds a tactile and intuitive aspect to the process. Together, these tools offer a comprehensive approach to the room analysis, enabling the home studio engineer to make informed decisions about acoustic treatment and monitor placement for optimal sound quality within the space.

Improvements to Experimental Design

To improve the analysis using a sine sweep generated in MATLAB, we could have improved our setup procedures ensuring that all equipment was properly calibrated before starting the sine sweep. To include, calibrating the output level of the signal and the sensitivity of the microphone to ensure accurate measurements. Minimize external noise such as the air conditioning unit and refrigerator. By incorporating these improvements into our experimental design, the accuracy and reliability of the room analysis could have been enhanced, leading to more effective acoustic treatment and a better-sounding home studio.

Appendix

Matlab Code

Part 1 Code

```
%Project1code.m
%Kevin Worley and Justin Roberts
%
clear; clc; close all;
%% Generating File for impulse signal a second in duration.
Fs = 48000;
impulse = [1 ; zeros(Fs,1)];
audiowrite('impulseSignal.wav', impulse, Fs);
%% Impulse Vs Delayed Impulse
[x, ~] = audioread('impulseSignal.wav');
[DImpulse, Fs] = audioread('ImpulseDelayPrint.wav');
DImpulseMono = DImpulse(:,1);
% N = length(x);
% M = length(DImpulseMono);
%
plot(x);
hold on
plot(DImpulseMono)
hold off
title('Impulse vs Delayed Impulse')
xlabel('Seconds')
xticks([0 12000 24000 36000 48000])
xticklabels({'0' '.25' '.5' '.75' '1'})
ylabel('Magnitude')
y = conv(DImpulseMono, x, 'full');
sound(y, Fs);
yreal = y(1:48001, 1);
plot(yreal);
plot(DImpulseMono);
Null = DImpulseMono - yreal;
plot(Null);
sound(Null, Fs);
title('Impulse vs Delayed Impulse')
xlabel('Seconds')
xticks([0 12000 24000 36000 48000])
xticklabels({'0' '.25' '.5' '.75' '1'})
ylabel('Magnitude')
audiowrite('convolvedDelayImpulse.wav', yreal, Fs)
%% Vocal vs Delayed Vocal
[Vocal, ~] = audioread('Vocal.wav');
```

```

[DVocal, Fs] = audioread(['VocalDelayFXPrint___.wav']);
DVocalMono = DVocal(:,1);
plot(Vocal);
hold on
plot(DVocalMono)
hold off
title('Vocal vs Delayed Vocal')
xlabel('Seconds')
xticks(0:48000:1200000)
xticklabels({1:1:25})
ylabel('Magnitude')
y = conv(DVocalMono,Vocal);
sound(y,Fs);
N = length(y);
M = length(DVocalMono);
yreal = y(N-M+1:end , 1);
% plot(yreal);
Null = yreal - DVocalMono;
audiowrite('convolvedDelayedVocal.wav', yreal,Fs)
%% Impulse EQ vs No EQ
[Impulse, ~] = audioread('impulseSignal.wav');
[ImpulseWet, Fs] = audioread('impulseEQPrint_1.wav');
WetMono = ImpulseWet(:,1);
a = fft(Impulse);
wet = fft(WetMono);
[Ha,Wa] = freqz(a,1,4096,Fs);
[Hwet,Wwet] = freqz(hNorm,1,4096,Fs);
semilogx(Wa,20*log10(abs(Ha))); axis([20 20000 -12 12]);
semilogx(Wwet,20*log10(abs(Hwet))); axis([20 20000 -12 12]);
figure;
freqz(a);
hold on
freqz(wet)
hold off
title('Impulse Eq vs No Eq')
xlabel('Seconds')
xticks([0 12000 24000 36000 48000])
xticklabels({'0' '.25' '.5' '.75' '1'})
ylabel('Magnitude')
y = conv(Impulse,WetMono);
sound(y,Fs);
yreal = y(1:51599 , 1);
Null = yreal - WetMono;
audiowrite('convolvedImpulseEQ.wav', yreal,Fs)
%% Vocal EQ vs Dry Vocal
[Vocal, ~] = audioread('Vocal.wav');
[VocalWet, Fs] = audioread('VocaleQFXPrint___.wav');
WetMono = VocalWet(:,1);

```



```

a = fft(Vocal);
wet = fft(WetMono);
figure;
freqz(a);
hold on
freqz(wet)
hold off
title('Vocal Eq vs Dry Vocal')
xlabel('Seconds')
xticks(0:48000:1200000)
xticklabels({1:1:25})
ylabel('Magnitude')
y = conv(Vocal,WetMono);
sound(y,Fs);
N = length(y);
M = length(WetMono);
yreal = y(N-M+1:end, 1);
null = yreal - WetMono;
audiowrite('convolvedVocalEQ.wav', yreal,Fs)

```

Part 2 Code

```

%ProjectPart2.m
%Kevin Worley and Justin Roberts
%
clear; clc; close all;
%
Fs = 48000;
Nyq = Fs/2;
Ts = 1/Fs;
t = [0:Ts:10];
x = chirp(t,0,10,22000)';
N = length(x);
%Generating a Fade-In and Fade-Out of 1 second each.
fadeSec = 1;
fadeSamples = fadeSec * Fs;
fadeIn = linspace(0, 1, fadeSamples)';
fadeOut = linspace(1, 0, fadeSamples)';
% Padding the fade arrays with ones to match the length of the signal
fadeIn = [fadeIn; ones(N - length(fadeIn), 1)];
fadeOut = [ones(N - length(fadeOut), 1); fadeOut];
% Applying fade-in and fade-out to the signal
fadeInSig = x .* fadeIn;
fadeInOutSig = fadeInSig .* fadeOut;
% Plotting the results
figure;
plot(t, fadeInOutSig);
title('Signal with Fade-In & Out');

```

```

xlabel('Seconds');
xticks(0:48000:480001)
xticklabels({1:1:10})
ylabel('Amplitude');
audiowrite('sinesweep.wav',fadeInOutSig,Fs);
% sound(FadeOutSig);
%%
%Steps for after Impulse collection
[x,Fs] = audioread('sinesweep.wav');
[y,Fs] = audioread('sinesweep_03.wav');
Sizex = length(x);
Sizey = length(y);
% Zero Padding x to size of y
padLength = Sizey - Sizex;
x = [x ; zeros(padLength,1)];
N = 16;
FreqX = fft(x);
FreqY = fft(y);
ConjX = conj(FreqX);
ConjY = conj(FreqY);
z = ConjY .* ConjX;
regularization = .5;
freqBin = zeros(N,1); %Pre-allocating freq array
HkBin = zeros(N,1); %Pre-allocating amp array
freq = 1/Nyq;
Hk = 0;
for n = 1:N
if z(n,1) == 0j
Hk = (z(n,1)/(z(n,1) + regularization)*Ts);
end
if z(n,1) > 0j
Hk = (z(n,1)/(z(n,1)*Ts));
end
freqBin(n,1) = freq; %Indexing sample into freq array
HkBin(n,1) = Hk; %Indexing sample info for Hk Bin.
freq = 2*freq; %Allocating new value for next octave in Hz.
end
hReal = real(ifft(HkBin));
% plot(hReal);
%Normalize Hn
hNorm = hReal/Fs;
plot(hNorm);
[H,W] = freqz(hNorm,1,4096,Fs);
semilogx(W,20*log10(abs(H))); axis([20 20000 -12 12]);
xlabel('Frequency (Hz)');
ylabel('Magnitude (dB)');
title('Room Responce');
grid on;

```

```

%% Analyzing IR for Hardware EQ
%Setting 1 Sweep
[Sweep] = audioread('sinesweep.wav');
[SweepSet1Wet, Fs] = audioread('HardwareEqSweep1Print_02.wav');
WetMono = SweepSet1Wet(:,1);
a = fft(Sweep);
wet = fft(WetMono);
figure;
freqz(a);
hold on
freqz(wet)
hold off
title('Sweep Eq vs Dry Sweep Setting 1')
xlabel('Seconds')
xticks(0:48000:480000)
xticklabels({1:1:10})
ylabel('Magnitude')
y = conv(Sweep,WetMono);
% sound(y,Fs);
N = length(y);
M = length(WetMono);
yreal = y(N-M+1:end, 1);
null = yreal - WetMono;
audiowrite('sweepWetSetting1.wav', yreal,Fs)
%Setting 2 Sweep
[Sweep] = audioread('sinesweep.wav');
[SweepSet2Wet, Fs] = audioread('HardwareEqSweepEq2Print_01.wav');
WetMono = SweepSet2Wet(:,1);
a = fft(Sweep);
wet = fft(WetMono);
figure;
freqz(a);
hold on
freqz(wet)
hold off
title('Sweep Eq vs Sweep Dry Setting 2')
xlabel('Seconds')
xticks(0:48000:480000)
xticklabels({1:1:10})
ylabel('Magnitude')
y = conv(Sweep,WetMono);
% sound(y,Fs);
N = length(y);
M = length(WetMono);
yreal = y(N-M+1:end, 1);
null = yreal - WetMono;
audiowrite('sweepWetSetting2.wav', yreal,Fs)
%Setting 1 Vocal

```

```

[Vocal] = audioread('Vocal.wav');
[VocalSet1Wet, Fs] = audioread('HardwareEQVocalSetting1Print_01.wav');
WetMono = VocalSet1Wet(:,1);
a = fft(Vocal);
wet = fft(WetMono);
figure;
freqz(a);
hold on
freqz(wet)
hold off
title('Vocal Eq vs Vocal Dry Setting 1')
xlabel('Seconds')
xticks(0:48000:1200000)
xticklabels({1:1:25})
ylabel('Magnitude')
y = conv(Vocal,WetMono);
% sound(y,Fs);
N = length(y);
M = length(WetMono);
yreal = y(N-M+1:end, 1);
null = yreal - WetMono;
audiowrite('vocalWetSetting1.wav', yreal,Fs)
%Setting 2 Vocal
[Vocal] = audioread('Vocal.wav');
[VocalSet2Wet, Fs] = audioread('HardwareEqVocalSetting2Print_01.wav');
WetMono = VocalSet2Wet(:,1);
a = fft(Vocal);
wet = fft(WetMono);
figure;
freqz(a);
hold on
freqz(wet)
hold off
title('Sweep Eq vs Sweep Dry Setting 2')
xlabel('Seconds')
xticks(0:48000:1200000)
xticklabels({1:1:25})
ylabel('Magnitude')
y = conv(Sweep,WetMono);
% sound(y,Fs);
N = length(y);
M = length(WetMono);
yreal = y(N-M+1:end, 1);
null = yreal - WetMono;
audiowrite('vocalWetSetting2.wav', yreal,Fs)

```